

Project Assignment: Build a Simple Product API

You are building an API for a small online shop. Visitors can view available products, but only authenticated users can add new products.

Use Python, Django, and Django REST Framework. No frontend or website design is required.

1. Create the project

Create a Django project named `shop_project` and an app named `products`.

Install and configure Django REST framework. Use Django's default SQLite database.

2. Create a Product model

Each product must contain:

- `id`: Automatically generated by Django.
- `name`: Product name, maximum 100 characters.
- `description`: A short description of the product.
- `price`: Product price, with up to two decimal places.
- `stock`: The number of items available.

Example product:

Name: Notebook

Description: A notebook with 100 pages.

Price: 120.00

Stock: 25

Create and apply database migrations to save the model structure.

3. Create a product serializer

Create a `ProductSerializer` using DRF's `ModelSerializer`.

Include all five product fields. The `id` must be generated automatically; users should not enter it.

The serializer must check that:

- The name is not empty.
- The price is greater than zero.
- The stock is a whole number and cannot be negative.

If submitted information is invalid, the API should return a useful error message.

4. Create an API to view products

Create this endpoint:

GET `/api/products/`

It should return the saved products, including each product's `id`, `name`, `description`, `price`, and `stock`.

Requirements:

- Anyone can view products without logging in.
- Return products ordered by id.
- If there are no products, return an empty list within the paginated response.

5. Create an API to add products

Use the same endpoint with a POST request:

POST /api/products/

An authenticated user should be able to submit:

```
{
  "name": "Notebook",
  "description": "A notebook with 100 pages.",
  "price": "120.00",
  "stock": 25
}
```

When the information is valid:

- Save the product in the database.
- Return the saved product, including its generated id.
- Return HTTP status 201 Created.

Hint: DRF's ListCreateAPIView supports both listing and creating products.

6. Add token authentication

Use DRF's built-in TokenAuthentication.

Requirements:

- Create a test user and generate an authentication token.
- Allow GET requests without a token.
- Require a valid token for POST requests.
- Use IsAuthenticatedOrReadOnly as the permission class.

When sending a POST request, include this header:

Authorization: Token YOUR_TOKEN

Replace YOUR_TOKEN with your test user's token.

A visitor who tries to add a product without a valid token must be denied access.

7. Test your project

Use Postman or another API testing tool. Complete these tests:

- View products without a token → the product list is returned.
- Add a valid product with a valid token → the product is created.
- Add a product without a token → access is denied.
- Add a product with an empty name → a validation error is returned.
- Add a product with a price of zero or less → a validation error is returned.

- Add a product with negative stock → a validation error is returned.
- View page two after adding six products → the sixth product is displayed.

For POST requests in Postman, select Body → raw → JSON and enter the product information.

9. Submission Guideline: Must submit a public GitHub repository link

- A short README explaining how to install dependencies, run migrations, start the server, and create a test user and token.
- Screenshots showing successful product viewing and creation.
- Screenshots showing one authentication error and one validation error.
- A screenshot showing the second page of products.

Completion goal: A visitor can browse products, an authenticated user can add valid products, and invalid product information is rejected. *Editing, deleting, shopping carts, and payments are outside this assignment.*